

MolmoB0T: Large-Scale Simulation Enables Zero-Shot Manipulation

Abhay Deshpande^{♥1*} Maya Guru^{♥1*} Rose Hendrix^{♥1*} Snehal Jauhri^{♥1,4*}

Ainaz Eftekhari^{♥1,2} Rohun Tripathi^{♥1} Max Argus^{♥1} Jordi Salvador^{♥1} Haoquan Fang^{♥1,2} Matthew Wallingford^{♥1} Wilbert Pumacay^{♥1} Yejin Kim^{♥1}

Quinn Pfeifer² Ying-Chun Lee² Piper Wolters¹ Omar Rayyan³ Mingtong Zhang⁵ Jiafei Duan^{1,2}
Karen Farley¹ Winson Han¹ Eli Vanderbilt¹

Dieter Fox^{1,2} Ali Farhadi^{1,2} Georgia Chalvatzaki⁴

Dhruv Shah^{♥†5} Ranjay Krishna^{♥†1,2}

¹Allen Institute for AI, ²University of Washington, ³University of California, Los Angeles, ⁴Technische Universität Darmstadt, ⁵Princeton University

*denotes equal contribution in alphabetical order, ♥ marks core contributors, and † indicates equal advising. See full author contributions [here](#).

Abstract



A prevailing view in robot learning is that simulation alone is not enough; effective sim-to-real transfer is widely believed to require at least some real-world data collection or task-specific fine-tuning to bridge the gap between simulated and physical environments. We challenge that assumption. With sufficiently large-scale and diverse simulated synthetic training data, we show that zero-shot transfer to the real world is not only possible, but effective for both static and mobile manipulation. We introduce **MolmoBot-Engine**, a fully open-source pipeline for procedural data generation across robots, tasks, and diverse simulated environments in MolmoSpaces. With it, we release **MolmoBot-Data**, a dataset of 1.8 million expert trajectories for articulated object manipulation and pick-and-place tasks. We train three policy classes: **MolmoBot**, a Molmo2-based multi-frame vision-language model with a flow-matching action head; **MolmoBot-Pi0**, which replicates the π_0 architecture to enable direct comparison; and **MolmoBot-SPOC**, a lightweight policy suitable for edge deployment and amenable to RL fine-tuning. We evaluate on two robotic platforms: the Franka FR3 for tabletop manipulation tasks and the Rainbow Robotics RB-Y1 mobile manipulator for door opening, drawer manipulation, cabinet interaction, and mobile pick-and-place. Without any real-world fine-tuning, our policies achieve zero-shot transfer to unseen objects and environments. On tabletop pick-and-place, MolmoBot achieves a success rate of 79.2% in real world evaluations across 4 settings, outperforming $\pi_{0.5}$ at 39.2%. Our results demonstrate that procedural environment generation combined with diverse articulated assets can produce robust manipulation policies that generalize broadly to the real world.

1 Introduction

Robotics foundation models are increasingly being built by a small number of well-resourced industrial labs. NVIDIA’s GR00T [1], Physical Intelligence’s π_0 [2, 3], and Google DeepMind’s Gemini Robotics [4] frames large-scale real-world training as the basis for generalist manipulation agents that act in the physical world. Despite their utility, much of what matters most for training such systems remains difficult for the broader

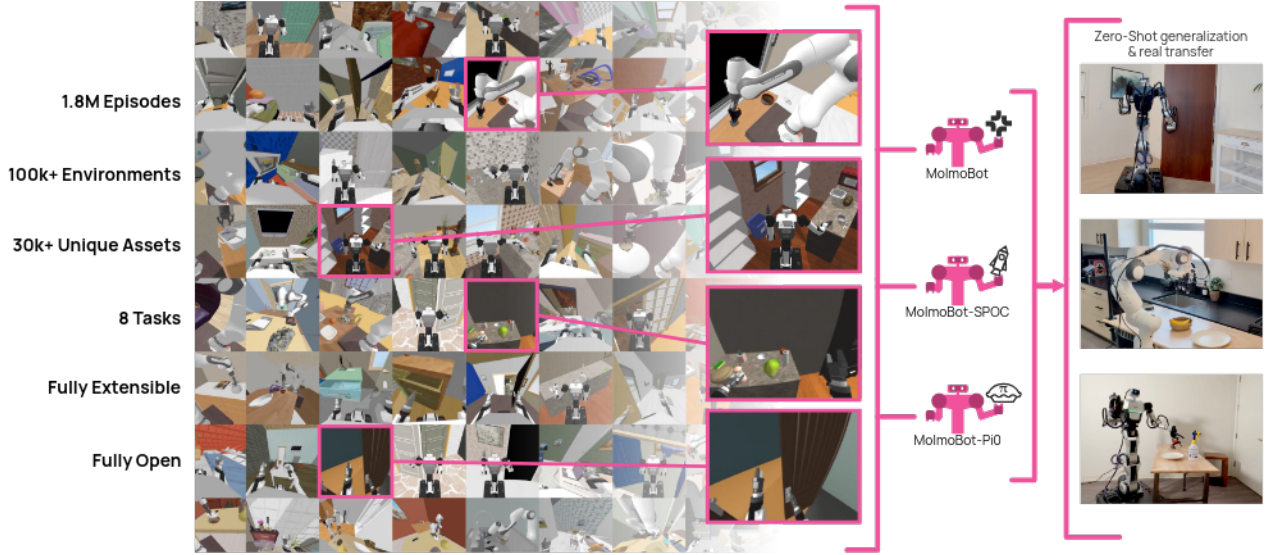


Figure 1 MolmoBot leverages diverse simulation data to achieve zero-shot sim-to-real transfer on multiple robotic tasks such as pick-and-place and door opening. This unlocks the ability to dramatically scale up the training data for generalist robotic foundation models.

community to study: the full data mixtures, collection processes, filtering decisions, scaling regimes, and training recipes behind the strongest models are often only partially disclosed. As a result, the knowledge of what it actually takes to build a robotics foundation model from scratch remains concentrated within a small set of institutional actors rather than broadly accessible to the field.

In the absence of open recipes for building these models end-to-end, much of the community has gravitated toward adapting existing systems rather than understanding the ingredients required to train them. This tendency is reinforced by a widely held assumption in robotics: that simulation alone is not enough for manipulation, and that the sim-to-real gap becomes manageable only after introducing some amount of real-world data for adaptation. Under this view, simulation is useful for pretraining, bootstrapping, or stress-testing, but not as a sufficient substrate for producing robust real-world manipulation policies on its own.

We challenge that assumption. We show that when simulation is scaled aggressively, across a diversity of environments, embodiments, articulated assets, and tasks, it can support zero-shot transfer to real-world mobile manipulation without any real-world fine-tuning, photorealistic rendering, or explicit domain adaptation.

This challenge arises from our prior work on navigation, SPOC [5]. SPOC showed that this tension can be overcome through scaled simulation data for navigation. Imitating shortest-path experts across hundreds of thousands of procedurally generated houses produces navigation policies that transfer zero-shot to real environments. A natural next question arises: can scaled simulated data enable zero-shot transfer for manipulation?

To study this question, we introduce **MolmoBot-Engine**, a fully open-source pipeline for procedural data generation across robots, tasks, and diverse simulated environments, and **MolmoBot-Data**, a dataset of 1.8 million expert trajectories spanning articulated object manipulation and pick-and-place. MolmoBot-Engine is built on top of a subset of our recently released MolmoSpaces [6], an ecosystem of 232k environments with 48k manipulable objects across 8 types of tasks. We procedurally generate robot trajectories across a variety of manipulation tasks, including tasks such as door opening, which requires whole-body manipulation.

Using this data, we train three policy classes. Our flagship model, **MolmoBot**, is built on top of Molmo2 [7], our video-language model capable of ingesting past frames for context. We augment this architecture with a DiT-based flow-matching action head that is *layerwise coupled* to the vision-language backbone. Each action layer cross-attends to the corresponding intermediate hidden states of the underlying VLM, while also incorporating robot-state features, allowing actions to be generated from multi-scale multimodal representations. Aside

from MolmoBot, we also train **MolmoBot-Pi0**, which exactly replicates the π_0 architecture for controlled comparison; and **MolmoBot-SPOC**, a lightweight non-VLA policy suitable for edge deployment and future RL fine-tuning.

We evaluate these policies on two robotic platforms: the Rainbow Robotics RB-Y1 mobile manipulator for door opening, drawer manipulation, cabinet interaction, and mobile pick-and-place, and the Franka FR3 for tabletop pick-and-place. Across both platforms, our policies transfer zero-shot from simulation to unseen real-world objects and environments, and outperform $\pi_{0.5}$ in our real-world evaluations. Specifically, on tabletop pick-and-place, our best MolmoBot achieves a success rate of 79.2% in real world evaluations across 4 settings while $\pi_{0.5}$ achieves 39.2%.

We provide ablations demonstrating the importance of data scale and diversity, and show through MolmoBot-Pi0 that our data yields strong performance even when the architecture is held constant. Our MolmoBot-Pi0 achieves a success rate of 46.7% in real world evaluations, improving upon $\pi_{0.5}$ at 39.2% when using the same architecture and training with MolmoBot-Data from scratch.

Broadly, our results suggest that the barrier to general-purpose manipulation may be less about an irreducible sim-to-real gap, and more about whether the community has access to sufficiently large, diverse, and open simulation pipelines for training robotics foundation models. We provide that access by open-sourcing all components.

2 Related Work

Imitation learning for manipulation. Imitation learning is the leading paradigm for robot manipulation. Initial methods focused on behavior cloning that map observations to actions [8, 9], while later work introduced hierarchical structures and temporal abstractions to address long-horizon tasks more effectively [10]. Recently, generative modeling techniques such as diffusion policies [11] have been introduced, demonstrating strong performance on manipulation benchmarks.

Recent developments have extended imitation learning to vision-language-action (VLA) models that integrate language understanding with perception and control within a unified architecture. Systems such as RT-1 [12] and RT-2 [13] showcase that increasing model capacity and utilizing multi-task robot datasets enable policies to perform hundreds of manipulation tasks conditioned on natural language instructions. More recently, π_0 and its subsequent variants [2, 3] applied a flow-matching action representation that enabled continuous action generation and supports generalist policies capable of cross-embodiment learning. Other recent works that explore cross-embodiment training using heterogeneous real-world robot datasets include X-VLA[14] that conditions a shared policy on embodiment-specific prompt tokens for multi-robot training, and LAP-VLA[15] that aligns robot control with languages by representing actions as language tokens. Although these systems exhibit impressive capabilities, they depend heavily on large-scale real-world robot demonstrations. In contrast, this work investigates training VLA policies exclusively from simulation-generated trajectories while preserving strong real-world performance.

Large-scale dataset and simulation. The advancement of generalist robot policies is closely associated with the availability of large-scale datasets. Several initiatives have gathered extensive real-world demonstrations spanning diverse tasks and embodiments, enabling learning from heterogeneous trajectories [16]. Datasets like DROID [17] offer large collection of manipulation demonstrations for training contemporary VLA models.

Owing to the high cost and logistical challenges of real-world data collection, recent research has increasingly emphasized simulation or synthetic datasets. GraspVLA [18] explores VLA policies trained on simulated grasping demonstrations, while the InternVLA family (InternVLA-M, InternVLA-A, InternVLA-H/N) [19] demonstrates large-scale pretraining for manipulation, action planning, navigation, and humanoid control using synthetic trajectories. Additionally, work such as PartInstruct [20] and Infinigen-Articulated [21] illustrates the effectiveness of procedurally generated simulation datasets in supporting robot learning research.

Our work extends this line of research by introducing MolmoBot-Engine, a fully open-source pipeline that enables scalable data generation in simulation across different robots, tasks, and diverse environments, and MolmoBot-Data, a large-scale generated dataset of expert manipulation trajectories. By combining procedural

scene generation with diverse rigid and articulated assets, our dataset enables training generalist policies that transfer to real-world deployment without any real-world demonstrations.

Articulated and mobile manipulation. Manipulating articulated objects such as doors, drawers, and cabinets remains challenging due to complex contact dynamics and partially observable object states. Mobile manipulation introduces additional complexity, requiring coordination among navigation, perception, and manipulation. Most large-scale manipulation systems concentrate on fixed-base manipulators operating in tabletop environments, where perception and workspace constraints are less complex [13, 17]. Several recent works that explore mobile manipulation typically address only a subset of the problem. For instance, some approaches focus on navigation relying on fixed-base manipulation skills for overall mobile manipulation tasks [22], or demonstrate only the feasibility of mobile manipulation platforms through real-world teleportation datasets [23] or real-world online adaptation strategies [24]. Other prior work has explored articulation-aware policies that incorporate object geometry and motion constraints. For example, FlowBot3D [25] learns manipulation flows to guide robot interaction with articulated objects.

Despite these advances, mobile manipulation remains underexplored within large-scale imitation learning frameworks. A recent work used simulations to collect a scalable dataset and demonstrated that sim-to-real transfer outperformed human teleoperators [26]. However, for particular articulated categories, such as door opening, solutions remain task-specific. This study evaluates policies on both a tabletop manipulator and a mobile manipulator that performs multiple tasks such as mobile pick-and-place and door opening. The results demonstrate that large-scale simulation-generated data can produce policies that generalize to both articulated and mobile manipulation scenarios without real-world demonstrations.

3 MolmoBot-Engine: A scalable manipulation data engine

We introduce **MolmoBot-Engine**, a procedural data generation pipeline for scalable robotic manipulation training, illustrated in Fig. 2. Our key insight is that manipulation policies benefit more from diversity across objects, configurations, and viewpoints than from photorealistic rendering. By rendering procedurally generated MolmoSpaces [6] environments in MuJoCo simulator with extensive domain randomization, we generate large-scale demonstration data at a fraction of the cost of real-world collection.

We note that MolmoBot-Engine is inherently constrained by the capabilities of the simulation platform. We focus on rigid body and articulated object manipulation (pick/pick-and-place and door/drawer/cabinet opening), as these are both tractable to model for modern simulators, as well as interesting and challenging tasks still unsolved by modern generalist policies. We hope this contribution can help towards extending simulation data generation to new classes of manipulation such as exceedingly contact-rich or soft-body manipulation.

3.1 MolmoSpaces environments and assets

We leverage the objects and scenes in MolmoSpaces [6], a large collection of procedurally generated indoor environments with realistic architectural variation, room layouts, and object placement, and individual rigid objects that can be procedurally added to any scene.

Environment setup. Each episode takes place in one of the more than 200k available pre-built MolmoSpaces scenes. The layout, furniture, and static objects remain fixed, but we can adapt every scene for specific tasks by sampling from a large pool of objects and placing task-relevant objects in suitable locations for each possible task specification (e.g., we can place objects to fulfill the role of receptacle targets, pickup targets, or just as additional distractors, for various manipulation tasks). Besides this, we can also randomize visual and physical parameters.

Asset sourcing. Rigid objects for pick-and-place tasks are sourced from iTHOR [27] and Objaverse [28], filtered for graspable size (placement receptacles with bounding boxes of side under 50 cm along the x and y axes and vertical size up to 15 cm, pickup objects with xy -plane diagonal less than that for the receptacle) and

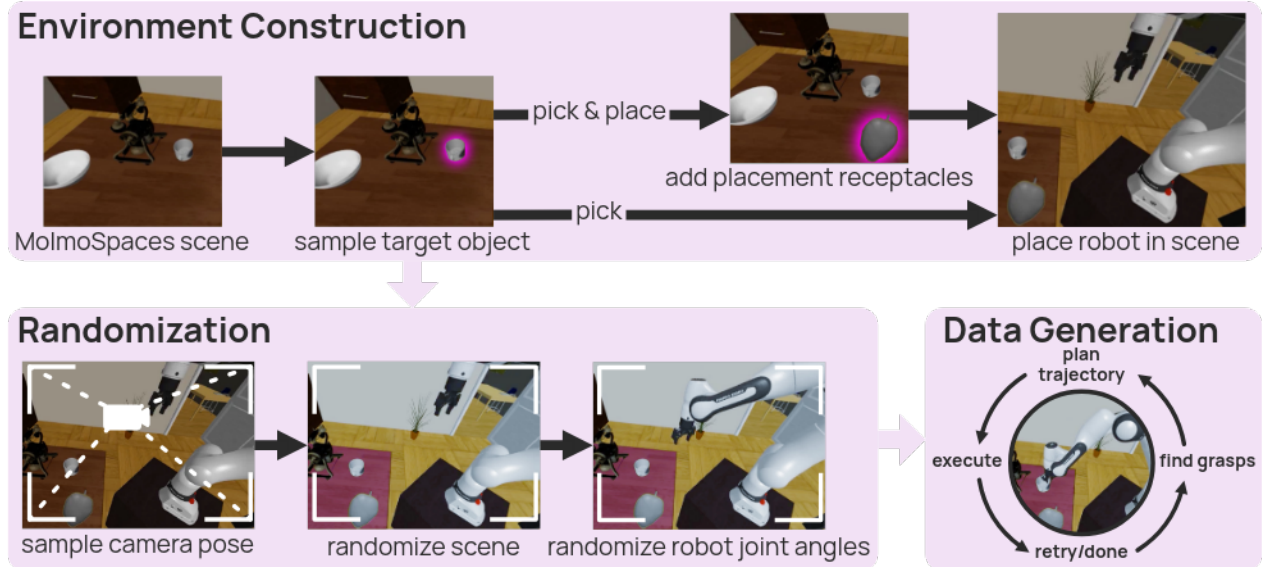


Figure 2 MolmoBot-Engine. Starting from a pre-built MolmoSpaces [6] house, we sample task-relevant objects, randomize visual and physical parameters, and iteratively replan as necessary until a successful trajectory is found.

watertight collider meshes. For task roles like the receptacle target in a pick-and-place task, we additionally ensure semantic relevance by filtering based on the object metadata provided by MolmoSpaces.

Domain randomization. We extensively perform domain randomization across three axes: environment randomization, action randomization (Sec. 3.2), and camera perturbation (Sec. 3.3.3). In addition to this, during model training we also perform image augmentation.

Focusing on **environment randomization**, after object placement, we randomize all visual and physical parameters supported by MuJoCo:

- **Lighting:** Number of lights ($[1-N]$), positions, intensities, colors, and shadow properties. We sample both point and directional lights to simulate diverse indoor conditions.
- **Textures:** Surface materials are randomized across placed objects and, where supported, existing scene elements. We sample from procedural textures and real-world texture maps sourced from AI2THOR assets [29].
- **Dynamics:** Friction coefficients, object masses, and joint damping are sampled within plausible ranges to encourage robust control policies.

Pose randomization. Manipulable assets are placed at randomized 6-DoF poses within the environment, subject to collision constraints and reachability from the robot’s workspace. We ensure diverse approach angles by sampling asset orientations relative to the robot base.

3.2 Robot configuration

We generate data for two robot platforms to enable both mobile manipulation and tabletop evaluation. Additional robot platforms can be easily added by future work.

Franka FR3. A 7-DoF Franka FR3 arm with a Robotiq 2F-85 parallel-jaw gripper, mounted on a fixed pedestal (0.58m height). We use the DROID [17] configuration to enable direct comparison with DROID-trained baselines and evaluation on existing benchmarks. Following DROID, data generation and evaluation are run at 15 Hz.

Rainbow RB-Y1. A mobile manipulator with a holonomic base (3-DoF: x, y, θ), a 6-DoF torso, a 2-DoF head (pan, tilt), and two 7-DoF arms, each equipped with a mechanically coupled parallel-jaw gripper. The base is controlled in planar joint-position mode; the head is passively set at initialization and not actuated during episodes.

Initial joint-configuration randomization. At episode initialization, each move group’s joint positions are sampled as $q_0 + \delta$, where q_0 is a nominal home configuration and $\delta_i \sim \mathcal{U}(-r_i, r_i)$ with per-joint noise magnitudes r_i . For both robots, the arm noise magnitudes are *graduated*: proximal joints receive smaller perturbations and distal joints larger ones. Concretely, the Franka arm uses $\mathbf{r}_{\text{arm}} = [0.025, 0.05, 0.075, 0.1, 0.125, 0.15, 0.175]$ rad (chosen via a Jacobian-weighted heuristic to bound TCP displacement to ≤ 10 cm), and each RB-Y1 arm uses $\mathbf{r}_{\text{arm}} = [0.05, 0.05, 0.075, 0.1, 0.125, 0.15, 0.175]$ rad. The RB-Y1 additionally randomizes head pan and tilt (± 0.2 rad $\approx \pm 11.4^\circ$ each) and gripper aperture (± 0.01 rad). Torso and base initial joint positions are not perturbed.

Action noise injection. During data collection, noise is injected into expert actions to prevent policies from overfitting to exact action replay. The noise is *action-proportional*: its standard deviation scales with the magnitude of the commanded displacement, so stationary commands receive no noise and large motions receive proportionally more.

For arm move groups, noise is applied in TCP space and then mapped back to joint space via the Jacobian pseudo-inverse. Specifically, we compute the commanded TCP displacement $\Delta \mathbf{x} = J \Delta \mathbf{q}$ from the Jacobian J and the joint-space command $\Delta \mathbf{q}$. Position noise is sampled from a truncated Gaussian with $\sigma_{\text{pos}} = \alpha \|\Delta \mathbf{x}_{\text{pos}}\|$ and clipped to ± 2 cm, where $\alpha = 0.1$ is a scale factor. Rotation noise uses $\sigma_{\text{rot}} = 0.1 \cdot \sigma_{\text{pos}}$, clipped to ± 0.1 rad ($\approx 5.7^\circ$). The resulting 6-DoF TCP noise vector ϵ_{tcp} is projected to joint space by solving $J \epsilon_q = \epsilon_{\text{tcp}}$ in the least-squares sense, and the noisy command is clipped to joint limits.

For the RB-Y1 base, planar noise is applied directly to (x, y, θ) commands using clipped Gaussians with $\sigma = 0.1 \cdot \|\Delta \mathbf{p}\|$, bounded to ± 2 cm in position and ± 0.05 rad ($\approx 2.8^\circ$) in heading. Action noise is disabled during simulated evaluation.

Gripper handling. Gripper close and open commands execute over fixed durations of 0.5 s and 0.25 s, respectively, followed by a settle period (`move_settle_time` = 0.1 s for the Franka; up to `max_grasping_timesteps` = 5 control steps for the RB-Y1) during which the arm is held stationary. This simulates real-world grasp settling time and ensures the object is stably grasped before subsequent arm motion resumes.

Camera pose. Per-episode perturbation to camera extrinsics is described in Section 3.3.

3.3 Sensor Configuration

After placing objects and applying domain randomization, we configure the robot’s sensors for the episode. We describe the camera systems for each platform, followed by additional sensor modalities.

3.3.1 FR3 camera system

The FR3 uses five cameras to provide diverse viewpoints for tabletop manipulation:

Wrist camera. A gripper-mounted camera analogous to a ZED Mini, with 52° vertical FOV ($\pm 4^\circ$ noise). Position is perturbed by ± 1.5 cm lateral, ± 0.5 cm vertical, and ± 2 cm in depth; orientation by $\pm 8^\circ$ in roll and $\pm 4^\circ$ in pitch and yaw.

Fixed shoulder camera. A robot-mounted exocentric camera positioned at a fixed offset from the robot base, with 71° FOV and light randomization (± 5 cm position, $\pm 8^\circ$ orientation). Placement is constrained to maintain visibility of task objects.

Randomized exocentric cameras. Three freely-placed cameras sample positions around the workspace center: two ZED2 analogues (64–72° FOV) and one GoPro analogue (137–140° FOV). For each camera, we sample distance (0.2–0.8m for ZED2, 0.2–0.5m for GoPro), height (0.05–0.6m above workspace), and azimuth (full 360°). Lookat target is the workspace center with ± 10 cm noise. Placement is rejected and resampled (up to 20 attempts) if task objects and gripper are not visible.

All FR3 cameras render at 624×352 , chosen to be close to the real-world resolution of 640×360 while keeping both dimensions a multiple of 16 for video encoding.

3.3.2 RB-Y1 camera system

The RB-Y1 uses three cameras matching the real robot’s sensor configuration:

Head camera. A head-mounted camera analogous to a GoPro in wide mode, rendered at 1024×576 and cropped to 768×576 (4:3 aspect ratio) in post-processing. We use a vertical FOV of 139° with $\pm 3^\circ$ noise. Position is perturbed by ± 1 cm in each axis, orientation by $\pm 4^\circ$ around each axis, and randomized fisheye warping is applied per-frame during training.

Wrist cameras. Left and right wrist-mounted cameras analogous to Intel RealSense D405 sensors. These render at 1024×576 (16:9 aspect ratio) with 58° vertical FOV and $\pm 4^\circ$ FOV noise. Position noise is ± 1.5 cm lateral, ± 0.5 cm vertical, and ± 1 cm in depth; orientation noise is $\pm 8^\circ$ in roll and $\pm 4^\circ$ in pitch and yaw. Depth is recorded for the benefit of future dataset utility but unused during training.

3.3.3 Proprioception and additional sensors

Beyond visual observations, we record proprioceptive state and auxiliary information for analysis and potential future use:

Robot state. We record the joint positions and velocities, TCP poses for each gripper, and the robot base pose.

Action labels. We record actions in multiple representations: commanded joint positions (absolute and relative to current joint positions), end-effector twist relative to current pose, and absolute end-effector pose. This enables training with different action parameterizations from the same trajectories.

Task state. We record the object start and goal poses, grasp state indicators, policy phase, and retry counts from the expert policy.

Camera parameters. We record the intrinsic and extrinsic parameters for each camera, enabling projection between 2D and 3D coordinates and potential depth-based augmentation. We also record points in the image frame on objects of interest in all cameras.

Depth images are recorded for RGB-D camera analogues but were not used in any of the training runs reported in this work.

3.4 Task definitions

Rigid object manipulation. We define four rigid-body manipulation tasks, each evaluated with both the stationary Franka FR3 and mobile RB-Y1 manipulators.

- *Pick:* Grasp a target object and lift it above its starting height. Success requires that the object is no longer supported by any non-robot surface and has been raised by at least 1 cm.
- *Pick-and-place:* Transport a target object to a specified receptacle. The task succeeds when at least 50% of the object’s weight is supported by the receptacle, and the receptacle has not been displaced by more than 10 cm or rotated by more than 45° .

- *Pick-and-place-next-to*: Place a target object adjacent to a reference object on the same surface. Success requires the surface-to-surface distance in the XY plane to lie within $[0, 25]$ cm and the reference object to remain within 15 cm of its initial position.
- *Pick-and-place-color*: Place an object on a receptacle identified by color (e.g., “place on the red plate”). Two receptacles identical (except for color) are placed in the scene; success criteria match *pick-and-place*.

Articulated object manipulation. We define two articulated-object tasks, evaluated with the mobile RB-Y1.

- *Open*: Open a nearby articulated object (e.g., cabinet, drawer, oven, dishwasher) to at least 15% of its joint range.
- *Open-door*: Open a nearby hinged door to at least 67% of its hinge joint range. The instruction is conditioned on the robot’s starting pose relative to the door, yielding either “push the door open” or “pull the door open.”

Language instructions. During training, each task episode is accompanied by a natural-language instruction whose referring expressions are sampled at episode initialization rather than fixed. For each object referenced in the instruction, we compute CLIP-based similarity scores between candidate referring expressions and all distractor objects in the scene, then sample an expression via a softmax distribution (temperature $\tau=0.02$) over the similarity-margin scores. This produces diverse yet unambiguous expressions (e.g., “the ceramic mug” vs. “the mug” depending on context). Expressions whose similarity margin falls below 0.03 or whose absolute target similarity is below 0.1 are filtered out to avoid ambiguous referrals. Further details on referral expressions are provided in Sec. A.2.

3.5 Expert planners

For each task, we generate expert demonstrations at scale via scripted demonstrators that iteratively sample grasps, verify feasibility, and execute motion for each task phase. Expert demonstrators for the Franka FR3 use IK-based interpolation, and for RB-Y1 use the CuRobo [30] motion generator to coordinate the many degrees of freedom with collision-aware motion planning.

Grasp sampling and filtering. Rather than assuming a fixed grasp pose, we load a large set of pre-computed grasp candidates per object from MolmoSpaces’ grasp dataset and progressively filter them:

1. **Candidate loading and ranking:** We load pre-computed 6-DoF grasps for each object, transform them into the world frame (including flipped variants), and rank them by a weighted cost that combines TCP proximity, rotation similarity, vertical alignment, and distance to the object center of mass.
2. **Collision filtering:** The top-ranked candidates are tested for gripper-scene collision by placing phantom collision bodies at each candidate pose in MuJoCo and running broadphase collision detection in batches of up to 128.
3. **IK feasibility:** Non-colliding candidates are checked for kinematic reachability via batch inverse kinematics (batches of up to 256). The highest-ranked feasible grasp is selected.

Phase-based trajectory generation. Each task is decomposed into a fixed sequence of phases, with motion planned independently per phase.

For *pick-and-place* tasks, the phases are: PREGRASP $\rightarrow$ GRASP $\rightarrow$ LIFT $\rightarrow$ PREPLACE $\rightarrow$ PLACE $\rightarrow$ POST-PLACE $\rightarrow$ STOW. For the RB-Y1 demonstrator, the PREPLACE and PLACE phases are combined, and we omit the additional STOW phase. In other words, the policy will first move to a pregrasp pose offset along the grasp approach axis, move to the grasp pose and close the gripper, lift the object, move to a pose above the receptacle before lowering to the placement pose and opening the gripper, and finally moving back to the home position.

Pick-and-place-next-to and *pick-and-place-color* are the same as *pick-and-place*, differing only in placement pose.



Figure 3 Expert demonstrations across multiple robots and manipulation tasks. Each row shows a trajectory conditioned on a language instruction. The top two rows illustrate Franka tabletop tasks (pick and pick-and-place), while the bottom rows show RB-Y1 mobile manipulation tasks (door opening and drawer opening). Columns visualize sequential frames from each trajectory.

Pick tasks proceed similarly to pick-and-place, but terminate after the LIFT phase.

For the *open* and *open-door* tasks, the phases are: PREGRASP $\rightarrow$ GRASP $\rightarrow$ ARTICULATE $\rightarrow$ POSTARTICULATE. After grasping the handle, articulation-specific end-effector waypoints are computed: a circular arc about the hinge axis for revolute joints (doors), or a linear path along the slide axis for prismatic joints (drawers). The planner solves for each waypoint sequentially using IK or trajectory optimization.

Retry behavior Each demonstrator is equipped with retry behavior. While executing a task, if the demonstrator detects a mistake (the object fell out of the grasp, the robot failed to acquire the grasp, etc.) it will reset to the first phase of the trajectory and try again. If more than 3 retries are triggered in an episode, it is terminated and discarded. This explicit retry behavior imbues policies with the ability to handle and recover from mistakes or disturbances.

Motion planning for RB-Y1 with CuRobo. For the RB-Y1, we use CuRobo [30] for GPU-accelerated collision-aware trajectory optimization. For each phase requiring collision-free motion (e.g., pre-grasp approach, placement), the planner constructs a cuboid-approximated collision world from the mesh-based MuJoCo scene, then plans in batches: multiple candidate goal poses are evaluated in parallel (default batch size of 4, up to 4

Table 1 MolmoBot-Data statistics by task. All episodes include RGB observations, proprioceptive state, action labels, and privileged information such as object visibility that the use of simulation affords.

Task	Robot	Episodes	Frames	Assets	Envs.	Avg. Length	Total length
Door-open	RB-Y1	101.6k	20.2M	–	17.0k	19.9 s	538 h
Open	RB-Y1	47.7k	7.1M	14.0k	10.7k	14.9 s	192 h
Pick	RB-Y1	64.3k	7.7M	4.3k	28.6k	10.7 s	185 h
Pick	Franka	786.7k	57.2M	10.8k	73.3k	4.8 s	1049 h
Pick-and-Place	RB-Y1	15.1k	2.5M	2.6k	9.5k	14.2 s	58 h
Pick-and-Place	Franka	558.4k	144.8M	7.4k	61.7k	17.1 s	2,632 h
PnP Next-To	Franka	182.9k	54.8M	9.5k	44.9k	20.1 s	1,020 h
PnP Color	Franka	28.8k	7.6M	3.3k	5.3k	17.3 s	138 h
Total		1.8M	301.9M	30.6k	104.5k	11.8 s	5,817 h

Table 2 Comparison to prior manipulation datasets. MolmoBot-Data provides substantially more episodes and environment diversity through procedural generation.

Dataset	Source	Episodes	Hours	Unique Envs.	Tasks	Embod.	Mobile Manip.
DROID [17]	Real	76k	350	564	86	1	✗
Open X-Embodiment [31]	Real	1M+	–	–	527	22	✓
AgiBot-World [32]	Real	1M+	2,976	100+	217	1	✓
RoboMimic [33]	Sim	~1k	–	1	5	1	✗
MimicGen [34]	Sim	50k+	–	–	18	1	✗
InternData-A1 [19]	Sim	630k	7,433	227	70	4	✗
RoboCasa-365 [35]	Sim	500k	2,200	2,500	365	1	✓
MolmoBot-Data (Ours)	Sim	1.8M	5,817	104.5k	8	2	✓

batches), and the trajectory with the least total joint displacement is selected. When a waypoint cannot be reached within a fixed number of control steps, the planner re-plans from the current configuration, up to a maximum of 5 re-planning attempts per phase. We provide additional details on CuRobo configuration in Appendix A.1.

3.6 Dataset Statistics

Table 1 summarizes MolmoBot-Data. We generate 1.8M episodes comprising 300M frames across 30k unique object assets and 100k procedural environment configurations.

Comparison to prior datasets. Table 2 compares MolmoBot-Data to prior manipulation datasets.

Generation throughput. A key advantage of simulation-based data generation is scalability. Using 100 NVIDIA A100 80GB GPUs, we generate approximately 1,024 episodes per GPU-hour, or equivalently more than 130 hours of robot experience per hour of wall-clock time. The full MolmoBot-Data dataset was generated in approximately 4,500 GPU-hours. This represents a near $4\times$ data throughput¹ compared to real-world data collection at equivalent scale with human demonstrators, enabling rapid iteration on data composition and task design as well as rapid adoption of new robotics platforms.

¹Using ALOHA [36] as reference, where the effective real-time factor of a single human demonstrator is 1/3 for tasks of similar duration to ours due to episode reset overhead or mistakes.

4 Models and training

We train three policy classes on MolmoBot-Data, enabling comparison across architectures and against external baselines.

4.1 MolmoBot: VLM-based manipulation policy

MolmoBot builds on Molmo2-4B [7], a vision-language model pretrained on large-scale image-text data. The architecture consists of three components: (1) a vision encoder that processes RGB observations from input camera views (2) a language model which jointly encodes visual features and task instructions, and (3) a DiT-based flow matching action head that predicts robot actions, as visualized in Fig. 4.

Vision encoder. Visual observations are encoded via SigLIP2 [37] and projected into the language model’s embedding space. We freeze the vision encoder and the projector weights during training and train only the action head and the language model. We train MolmoBot to ingest up to $F = 3$ frames per view. We encode each image individually and image tokens for each 2×2 patch windows are pooled into a single vector using a multi-headed attention layer, where the mean of the patches serves as the query. Each image is encoded with 192 tokens. We concatenate image tokens from available camera views (head-mounted, external, and wrist cameras, depending on the platform), interleaved with text tokens encoding the image indices and view indices when appropriate. Optionally, we encode the corresponding initial-timestep images to provide context about the starting scene configuration.

LLM. The LLM takes as input the visual tokens interleaved with image indices jointly with the tokenized language instruction. For tasks requiring spatial grounding, we optionally condition on 2D point coordinates specifying target objects or placement locations; these are injected as special tokens in the instruction stream (e.g., `<point coords=> OBJECT </points>`). We use bi-directional attention for the vision tokens and causal attention for the text tokens during training and inference.

Action head. The action head is a DiT [38] which contains self-attention and cross-attention in each layer, where it attends to features of the Molmo2 backbone via cross-attention. Following recent work on flow matching for action prediction [2], the DiT iteratively denoises action chunks conditioned on a continuous timestep embedding $t \in [0, 1]$. The timestep embedding is used by each DiT block to modulate the embedding via adaptive layer normalization [3].

MolmoBot’s action head has the same number of layers as the LLM encoder, and each action layer cross-attends to the hidden states of the input sequence (including the encoding of both vision and language) of the corresponding LLM layer. LLM and DiT have different hidden dimensions, so hidden states from the LLM are also projected to DiT’s hidden dimension. We also encode robot states through a single-layer MLP, and concatenate them to the end of the VLM sequence before entering cross-attention at each layer. We train the action head to predict chunks of $H = 16$ actions and execute 8 before re-querying the policy following [36].

Action representation. We parameterize actions in joint space using two representations: absolute joint positions and joint position deltas. Both are continuous values representing the target configuration for each joint. At each timestep, the policy predicts targets for all actuated joints, including the gripper. For the RB-Y1’s mobile base, we additionally predict base velocity commands (linear and angular) which are concatenated to the joint action. Joint-space control avoids the computational overhead and potential singularities of inverse kinematics at execution time. We train separate model variants on absolute and delta representations for the Franka FR3 task and compare their performance in Section 5. We only use delta policies for training the mobile manipulation task.

Single-frame training. We train MolmoBot with the behavior cloning objective. We train with a batch size of 1024 and train the model for 200K steps for the static manipulation task and for 100K steps for the mobile manipulation task. We use a learning rate of $1 \cdot 10^{-5}$, using a $2k$ step warm up for the LLM and a 200 step warm up for the action head. When sampling training examples from an expert roll-out, we up-sample steps with retry grasping behavior by $3\times$, steps with a successful pick by $2\times$ and task completion behavior by $2\times$.

The motivation is to improve the model’s grasping behavior and to avoid picking objects after completing its task.

Our action head has a significantly lighter compute footprint than the VLM encoder. We leverage this during training by sampling multiple time steps T per example to denoise in parallel. This enables us to train the model at multiple time steps for a given observation and action pair. This in turn improves the convergence and the accuracy of the model. We use $T = 8$ to train all MolmoBots unless otherwise stated and report performance with various settings in section 5. We denote the single frame model MolmoBot-Img.

Multi-frame training. We train two multi-frame MolmoBots denoted as MolmoBot ($F=2$) and MolmoBot ($F=3$) with number of frames $F = 2$ and $F = 3$ respectively. For the multi-frame training, we initialize the model with the weights from MolmoBot-Img and train the model for 50K steps while keeping all the other training details the same as MolmoBot-Img. When using multiple frames, the model takes as input the frame from the cameras at the current state and the frames sampled D steps ago. We use $D = 8$ in all our experiments. Practically, the $F = 3$ model takes the current state, the state ~ 0.5 second before the current state and the state ~ 1 second before the current state.

4.2 MolmoBot-Pi0

To isolate the effect of MolmoBot-Data on real-world VLA performance, we present MolmoBot-Pi0, a VLA with identical architecture as π_0 [2], trained entirely on our synthetic data from the initial Paligemma weights. This enables head-to-head comparisons with existing SOTA VLAs, controlling for modeling or architecture changes.

Architecture. Following [2], MolmoBot-Pi0 uses the Paligemma 3B VLM with a flow-matching action expert. We use the openpi [39] codebase for all MolmoBot-Pi0 modeling code, ensuring equivalence with π_0 .

Training protocol. We train for 200k steps at a batch size of 1024 with a learning rate of $5 \cdot 10^{-5}$, using a 1k step warmup. To prevent overfitting to simulation rendering artifacts, we freeze the entirety of the SigLIP vision encoder. Robot actions are supervised as absolute joint positions, following findings from PolaRiS [40]. All other training parameters (flow matching timestep sampling, other optimizer hyperparameters, etc.) are left as the default values.

4.3 MolmoBot-SPOC: A lightweight transformer policy

Overview. SPOC [5] is a transformer-based architecture that demonstrated that imitation learning from shortest-path experts across hundreds of thousands of procedurally generated houses can produce navigation policies that transfer zero-shot to real-world environments. Inspired by this architecture, MolmoBot-SPOC introduces a lightweight transformer-based policy with several modifications that make it suitable for our static and mobile manipulation tasks.

Visual, Language, and Proprioceptive Encoding Visual observations from all camera inputs are encoded using a SigLIP2-Base patch 16/256 image encoder [37], retaining the full set of patch tokens. Language goal instructions are encoded separately using the SigLIP text encoder [41]. The resulting token sequences consist of (1) visual patch embeddings, (2) language goal tokens, and (3) the robot’s current joint state projected into the model’s token dimension via a learned linear projection. These tokens are concatenated along the sequence dimension to form the cross-attention memory of the action decoder (Fig. 4). For tasks that provide spatial goal specifications, MolmoBot-SPOC optionally incorporates point-based goal encodings into the cross-attention memory. Depending on the task, one or two 2D pixel coordinates are provided: a single normalized image coordinate (x, y) for *pick*, *open*, and *door-open* tasks, or two coordinates (x_1, y_1, x_2, y_2) for *pick-and-place* tasks. Each coordinate is first passed through a sinusoidal positional encoder and then projected into the model’s token dimension using a linear layer. A learned coordinate position embedding is added to each encoded point, and the resulting point tokens are concatenated with the other inputs in the cross-attention memory. MolmoBot-SPOC does not condition on any trajectory history; only the current

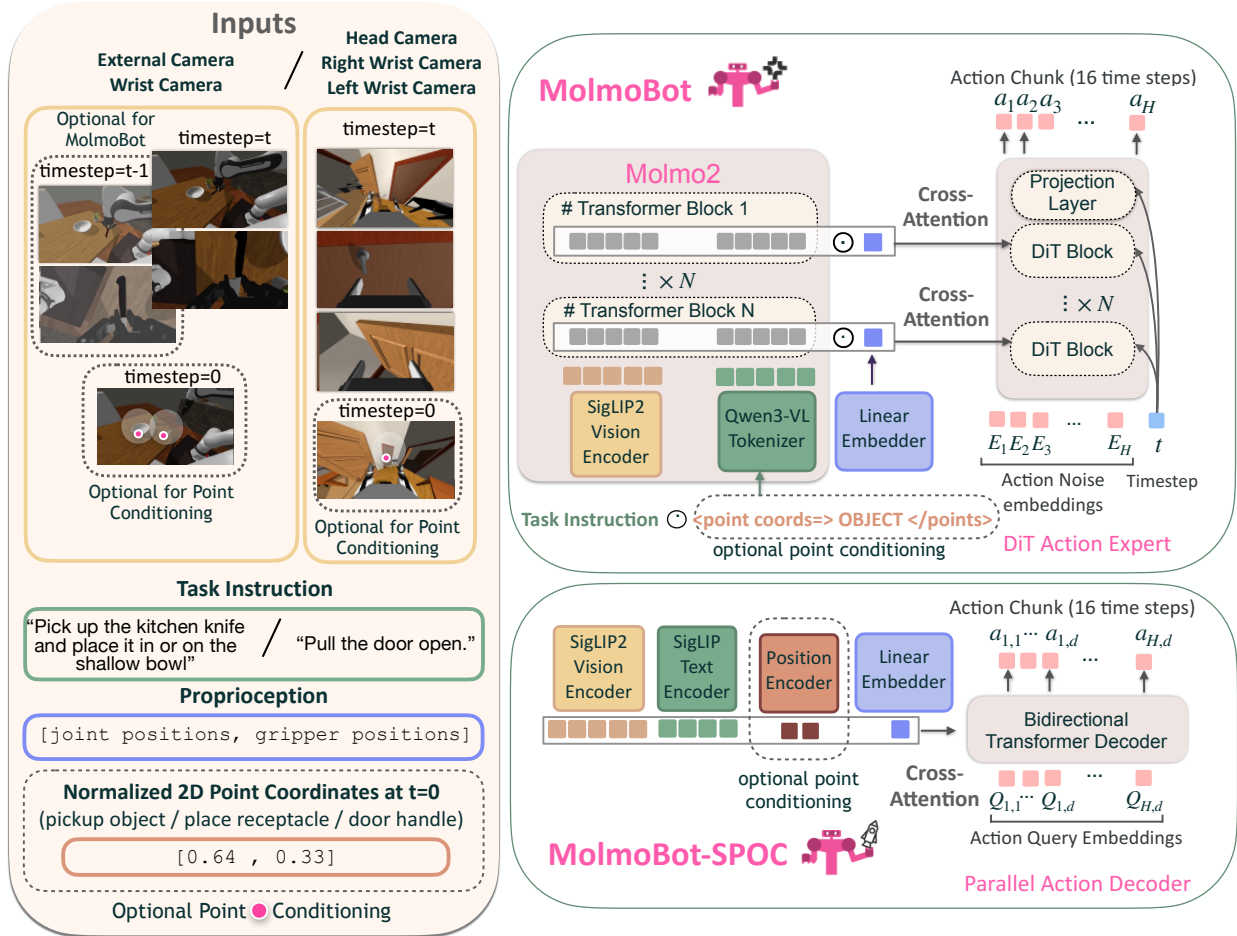


Figure 4 Policy architectures. We train three policy classes on MolmoBot-Data. **Left:** Input observations include RGB images from multiple camera views at the current (and optionally initial) timesteps, proprioceptive state, a language task instruction, and optional 2D point conditioning for specifying target objects or locations. **Top right:** MolmoBot uses a Molmo2 vision-language backbone with a DiTX-based flow matching action head that attends to visual features via cross-attention and predicts action chunks of 16 timesteps. **Bottom right:** MolmoBot-SPOC uses SigLIP2 vision and text encoders with a bidirectional transformer decoder that processes learned action query embeddings to predict actions in parallel. Both architectures support optional point conditioning. MolmoBot-Pi0 (not shown) exactly follows the π_0 architecture [2] to enable controlled comparison.

timestep’s observations and state are used. Optionally, we encode the corresponding initial-timestep images to provide context about the starting scene configuration when using point-based goal specification.

Action representation and quantile binning. MolmoBot-SPOC formulates action prediction as a discrete classification problem. Continuous action values are tokenized using a quantile binning strategy. Prior to binning, actions are normalized using the 1st and 99th percentiles of the training distribution, rescaling and clipping values to the $[-1, 1]$ range based on empirical quantiles. The normalized action space for each dimension is then divided into 256 bins, where bin boundaries correspond to equally spaced quantiles of the data (i.e., the $k/256$ quantile for $k = 1, \dots, 256$). This produces data-adaptive bins that are approximately uniformly populated, yielding a well-calibrated discrete representation of the continuous action space. The decoder predicts a categorical distribution over the 256 bins independently for each action dimension at every timestep in the chunk and is trained using a standard cross-entropy loss.

Parallel action decoding. Following [36], MolmoBot-SPOC replaces the autoregressive decoder used in SPOC with a non-causal parallel decoder (Fig. 4). Instead of predicting actions sequentially, the decoder predicts an

Table 3 Multitask data mixture for all MolmoBot Franka FR3 policies. The data mix is selected to ensure all coverage of each of the individual task’s training set.

Task	Sampling Ratio
Pick	20%
Pick-and-place Fixed Height	10%
Pick-and-place Random Height	35%
Pick-and-place-next-to	20%
Pick-and-place-color	15%

entire chunk of $D \times T$ action tokens in a single forward pass, where D is the number of robot action dimensions and $T = 16$ is the fixed chunk length. The decoder is provided with $D \times T$ learnable query embeddings—one for each (action dimension, timestep) pair in the chunk. Using bidirectional self-attention allows each query token to attend to all others within the chunk. Temporal structure is encoded using sinusoidal positional encodings applied over the flattened sequence of $D \times T$ positions, which are added to the learnable query embeddings before decoding.

4.4 Implementation details.

Data mixing Table 3 details the different training sets we use to train all the Franka FR3 policies. Pick-and-place Random Height comprises of pick and place tasks with the robot position initialized at random heights, while Pick-and-place Fixed Height initializes the model at the default droid position [17]. Training with randomized heights makes the model more robust to inference time variations. Pick-and-place-color trains the model to attend to the color attribute in input task and Pick-and-place-next-to training helps improve the models spatial understanding. Table 4 details the various data mixtures used to train RB-Y1 policies. Door-open is a specialized door opening task, while the Open task includes training samples to open cabinets and drawers.

We use image augmentation to train the model improve our models sim to real transfer. Specifically, we use ColorJitter, GaussianBlur, RandomPosterize, RandomSharpness and RandomGrayscale with different probabilities. We add prompt randomization during training to make the robot robust to inference time variation in language instruction.

5 Experiments

To illustrate the performance and generality of MolmoBot policies, we evaluate on multiple tasks in various real-world and simulated settings.

We begin with real-world evaluations, demonstrating the policies’ strong zero-shot sim2real transfer in multiple settings. We further corroborate these results with diverse simulation evaluations, including on an established manipulation benchmark. Finally, we ablate key design and data-mixture decisions and analyze the recipe for sim2real transfer.

Crucially, all models are only trained on sim, with zero task-specific or real-world post-training or finetuning.

Table 4 Multitask data mixture for all MolmoBot and MolmoBot-SPOC RB-Y1 policies.

Model	Open	Door-open	Pick	Pick-and-place
MolmoBot Multitask	20%	20%	30%	30%
MolmoBot Door Specialist	-	100%	-	-
MolmoBot-SPOC Rigid	-	-	50%	50%
MolmoBot-SPOC Articulated	45%	55%	-	-

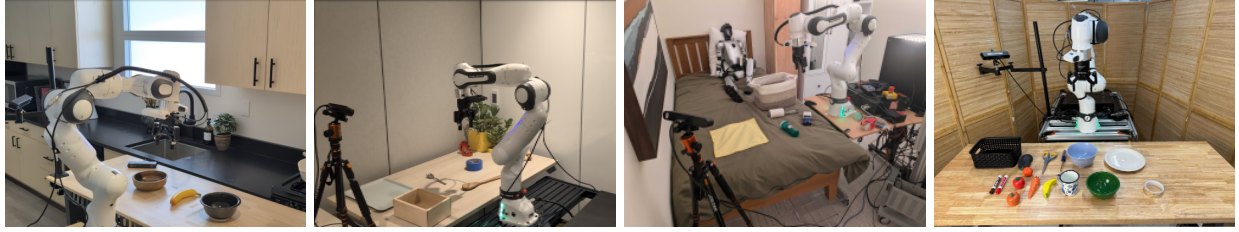


Figure 5 Real-world environments for our DROID evaluations. From left to right: kitchen, workroom, bedroom, office. Additional details in the Appendix.



Figure 6 Real-world environments for our RBY1 articulated and rigid object mobile manipulation evaluations.

All MolmoBot policies have never seen any real-robot data.

5.1 Zero-Shot Real-World Transfer

5.1.1 Baselines

DROID For our DROID evaluations, we compare against π_0 -DROID [2] and $\pi_{0.5}$ -DROID [3], which are SOTA open-weights manipulation policies for DROID. Both are trained with $>10k$ hours of real-world manipulation demonstrations, and $\pi_{0.5}$ further improves by adding new innovations like subgoal prediction, heterogenous cotraining, and more. Therefore, π_0 is the most closely comparable baseline in terms of modeling decisions, while $\pi_{0.5}$ represents one of the best generalist manipulation policy available today. Note that unlike MolmoBot-Data, the data used to pretrain the π -family models is not publicly available, limiting future reproduction or study by the field at large.

RB-Y1 The RB-Y1 has significantly less community adoption than DROID. MolmoBot policies are therefore, to our knowledge, the first generalizable pick-and-place and articulated object manipulation policies available for the RB-Y1. Lacking other generalist baselines, we reserve quantitative comparisons for our DROID evaluations.

5.1.2 Static Manipulation Evaluation

Setup We evaluate MolmoBot policies and baselines using three different physical DROID platforms, in four real-world environments across two geographical locations and institutions. In each environment, we evaluate each policy on 10 pick-and-place tasks, with 3 trials each. Cumulatively, each policy is evaluated 120 times. Evaluation environments are pictured in Fig. 5, and further details on real-world environment and task design are provided in Sec. A.3 in the appendix.

For pick-and-place tasks, given a task prompt (e.g. “put the banana in the black bowl”), the policy must move the specified object to be stably in or on the given receptacle. If the policy accomplishes this for a reasonable amount of time within the episode horizon (900 steps), the trial is counted as a success. Failure to do so within the episode horizon, or exhibiting unsafe behavior (high-speed collisions, etc.) before success counts as a task failure.

Results In our real-world static manipulation evaluations, MolmoBot policies exhibit strong zero-shot sim2real transfer, as illustrated in Fig. 7. MolmoBot and MolmoBot-Img significantly outperform $\pi_{0.5}$ -DROID while MolmoBot-Pi0 is competitive, all without the benefit of $\pi_{0.5}$ -DROID’s architectural improvements.

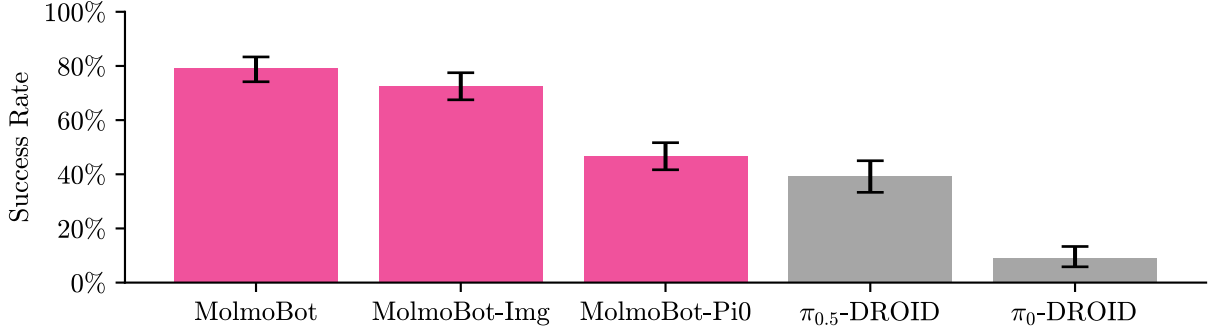


Figure 7 MolmoBot policies exhibit strong zero-shot sim2real performance across our real-world DROID evaluations, outperforming SOTA policies trained on large-scale real-world demonstrations. Bar heights reflect mean success rate and error bars represent 95% confidence intervals, estimated via stratified bootstrapping. Here, MolmoBot denotes the MolmoBot (F=2) variant.

Additionally, all MolmoBot policies perform much better than π_0 . Despite having identical architectures, MolmoBot-Pi0 significantly outperforms π_0 in our evaluations. Therefore, this difference in performance can only be explained by data.

This suggests that the diversity of simulated demonstration data is sufficient to deliver performance on-par or better than commensurate amounts of real-world data, the diversity of which is limited by real-world cost and practicality. We further study to what extent different types of data diversity and scale impact policy performance in Sec. 5.3.

5.1.3 Mobile Manipulation Evaluation

Setup We evaluate the MolmoBot Door Specialist policy on a door opening task in three real-world environments, each featuring a distinct pull door with different visual textures, handle configurations, and surrounding scene context. Unlike push doors, pull doors require the robot to precisely grasp the handle and exert a pulling force, making the task significantly more challenging — the robot cannot rely on contact-rich recovery strategies or simply driving into the door to produce motion. Each environment also features distinct wall geometry and background clutter, testing the generalization of the policy across varied visual conditions. Episodes were executed at a 100ms inference timestep for safety reasons, which is slower than the timestep used during simulation evaluation.

For our simulation results, we evaluate both MolmoBot and MolmoBot-SPOC across four tasks. *Pick* and *pick-and-place* are evaluated in the MSProcObj scene dataset, while *open* is evaluated in the MSProcCrafted dataset, and Door Open in the MSProc dataset. *Pick*, *open*, and *door-open* benchmarks consist of 2,000 episodes each, while *pick-and-place* uses 1,000 episodes. We report the oracle success rate, where an episode is considered successful if the task reports 5 consecutive successful steps at any point during the trajectory. Simulation evaluation was run with an inference dt of 800ms; in other words, we execute 8 of the predicted actions from a given action chunk where each action has a dt of 100ms.

Results In the real results represented by Tab. 5, we observed 4 out of 9 trials with handle grasp success and 2 out of 9 trials with door opening success. A recurring source of failure across Door 1 and Door 3 was difficulty grasping the handle. Both of these doors have handles positioned on the right side of the door, a configuration that is underrepresented in typical door interaction datasets and in our training data, which may explain the policy’s reduced grasping reliability in these cases. In contrast, Door 2, whose handle configuration was more commonly represented, saw successful grasps in all three trials.

Several trials were also affected by hardware faults, where the robot triggered its own emergency stop at various stages of execution. Once the e-stop is activated, the gripper cannot be reset mid-episode, meaning that if a fault occurs during the grasp phase the robot is unable to recover and the episode fails regardless of

Table 5 Door opening task results. Each door has a distinct visual texture. Trials differ in robot base position.[†]Hardware fault occurred during trial.

Door	Trial	Grasp	Opened	Failure Mode
1	1	✓ [†]	✗	HW fault during grasp phase
	2	✗	✗	Base collision
	3	✗	✗	Joint limit reached
2	1	✓ [†]	✗	HW fault during grasp phase
	2	✓	✓ [†]	HW fault during opening phase
	3	✓	✓ [†]	HW fault during opening phase
3	1	✗	✗	Base collision
	2	✗ [†]	✗	HW fault during door approach
	3	✗	✗	Incorrect gripper orientation

subsequent behaviour. In trials where faults occurred during the opening phase (Door 2, Trials 2 and 3), the robot had already successfully grasped the handle and was able to complete the door opening despite the fault, suggesting that the policy had committed to a successful trajectory before the fault manifested.

5.2 Simulation Evaluation

In Sec. 5.1, we demonstrated that MolmoBot policies demonstrate strong performance in real-world evaluations. However, real evaluations are expensive, and therefore have relatively large associated uncertainty and reduced controllability. To address this, we conduct systematic simulation evaluations, providing greatly reduced uncertainty, making trends more clearly visible, and enabling thorough data-mixing ablations.

Setup. We evaluate on held-out procedural houses with asset instances unseen during training, following an evaluation protocol similar to MolmoSpaces [6]. For each task, we generate evaluation episodes across a large number of held-out houses and report the oracle success rate (task completion at any timestep). Full details on evaluation tasks are provided in section A.4.

Tasks. We evaluate on a progression of increasingly difficult tasks (table 6). We begin with a simple pick task in a controlled configuration (*Pick MSProc*, 1000 episodes). The next set of tasks (200 episodes each) introduces more challenging object and viewpoint distributions in three variants: standard MuJoCo rendering (*Pick Classic*), photorealistic filament rendering (*Pick*) which is out of distribution for our training data, and heavily randomized camera viewpoints (*Pick Random-Cam*). Pick tasks are allotted 20 seconds. We then evaluate pick-and-place variants including placing objects inside a receptacle (*Pick&Place*), next to a target (*PnP Next-To*), and in a receptacle of a specified color (*PnP Color*), all using filament rendering. Pick-and-place tasks are allotted 40 seconds due to their increased difficulty. We report oracle success (task completed at any timestep) for pick tasks, as termination behavior is not well-defined for object lifting.

Baselines. We compare against several existing vision-language-action models. $\pi_{0.5}$ [3] is evaluated both zero-shot and after fine-tuning on MolmoBot-Data for 15K steps in order to adapt it to simulation. We also evaluate StereoVLA [42], LAP-VLA [15], and X-VLA [14] zero-shot.

Results. Our models substantially outperform all baselines across tasks. On the controlled *Pick MSProc* task, MolmoBot (F=2) achieves 92.0% success compared to 47.0% for the strongest baseline ($\pi_{0.5}$ -Finetune). The gap widens on more challenging distributions: on *Pick Random-Cam*, our models achieve 60–66% success while $\pi_{0.5}$ variants reach only 13–29%. Other VLA baselines (StereoVLA, LAP-VLA, X-VLA) fail almost entirely on our evaluation suite, with success rates below 6% on most tasks.

On pick-and-place tasks, which require both grasping and placement, MolmoBot variants achieve 60–67% success on *Pick&Place*. $\pi_{0.5}$ -Finetune achieves 46% on this task. Notably, our models generalize to compositional

Table 6 Evaluation on simulation held-out environments and real robot episodes. Simulation success rates are evaluated over 200 episodes per task, except Pick-MSPProc which uses 1000 episodes. Real robot evaluations are done over 120 episodes. All models evaluated zero-shot in real without task-specific finetuning.

Model	Pick MSPProc	Pick Classic	Pick	Pick Random-Cam	Pick&Place	PnP Next-To	PnP Color	Avg.	Real
$\pi_{0.5}$ [3]	36.4	9.3	14.0	12.8	20.0	35.5	16.2	20.6	31.3
$\pi_{0.5}$ -Finetune	47.0	20.5	24.0	28.5	46.0	36.9	53.0	36.6	—
StereoVLA [42]	6.3	4.3	1.1	N/A	0	0	0	—	—
LAP-VLA [15]	12.6	0.5	0.5	0	0	0	0	1.9	—
X-VLA [43]	6.0	5.8	0.9	0.2	0	0	0	1.8	—
MolmoBot-Pi0	65.5	30.5	32.0	36.0	41.5	41.8	42.5	41.4	46.7
MolmoBot-Img	92.8	67.0	63.0	62.5	67.0	47.5	65.0	66.4	72.5
MolmoBot (F=2)	92.0	63.0	66.0	61.0	60.0	55.0	68.0	66.4	79.2
MolmoBot (F=3)	91.9	61.0	63.5	60.0	62.5	46.5	65.5	64.4	75.0

Table 7 Simulation and real evaluation with restricted camera setup. Success rate averaged over 1000 episodes in simulation and 30 tasks in a real-world kitchen. All models evaluated zero-shot without task-specific finetuning.

Model	Pick MSPProc (sim)	Pick Kitchen (real)
StereoVLA [42]	6.3	—
LAP-VLA [15]	12.6	—
π_0 [2]	16.2	20.0
$\pi_{0.5}$ [3]	36.4	63.3
$\pi_{0.5}$ -Finetune	47.0	—
DreamZero [44]	52.1	—
MolmoBot-Pi0	65.5	53.3
MolmoBot-SPOC	70.4	36.6
MolmoBot-Img	92.8	86.6
MolmoBot (F=2)	92.0	70.0
MolmoBot (F=3)	91.9	73.3

instructions (*PnP Color*) where they must identify correct receptacle by color, achieving 38–41% final success.

Averaging across simulation tasks, MolmoBot (F=3) achieves 64.4% compared to 31.3% for $\pi_{0.5}$ zero-shot. MolmoBot-Pi0, which uses the π_0 architecture trained on our data, achieves 46.7%—substantially higher than $\pi_{0.5}$ zero-shot and competitive with $\pi_{0.5}$ -Finetune on pick tasks—demonstrating that much of the performance gain comes from MolmoBot-Data rather than architectural differences.

On real-world evaluation, MolmoBot (F=3) achieves 75.0% success, compared to 31.3% for $\pi_{0.5}$.

Table 7 evaluates models using only the fixed-shoulder camera, a more constrained setup that matches typical single-camera deployments. We test in simulation (*Pick MSPProc*, 1000 episodes) and on 30 real-world trials in a kitchen environment (*Pick Kitchen*).

In simulation, our models maintain strong performance: MolmoBot variants achieve 91–93% success, while MolmoBot-SPOC reaches 70.4%. $\pi_{0.5}$ -Finetune achieves 47.0% and $\pi_{0.5}$ zero-shot 36.4%. StereoVLA and LAP-VLA again perform poorly (6.3% and 12.6%).

On real-world evaluation, our models demonstrate successful zero-shot sim-to-real transfer. MolmoBot (F=3) achieves 73.3% success and MolmoBot (F=2) achieves 70.0%, compared to 63.3% for $\pi_{0.5}$ zero-shot. MolmoBot-SPOC achieves 36.6%, lower than the VLA variants but notable given its substantially smaller size and suitability for edge deployment. These results confirm that policies trained entirely on MolmoBot-Data transfer to real environments without fine-tuning.

Table 8 Simulation evaluation for RB-Y1 policies on held-out environments. All models evaluated zero-shot without task-specific finetuning.

Model	Pick	Pick & Place	Open	Door Open
MolmoBot Multitask	44.8%	22.5%	25.2%	70.2%
MolmoBot Door Specialist	–	–	–	77.7%
MolmoBot-SPOC Rigid	10.5%	1.8%	–	–
MolmoBot-SPOC Articulated	–	–	21.8%	58.8%

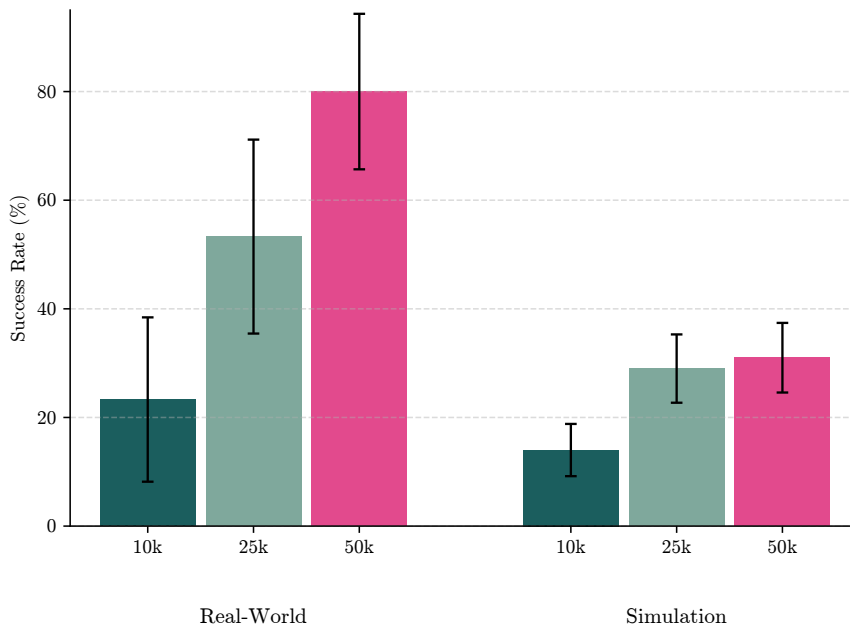


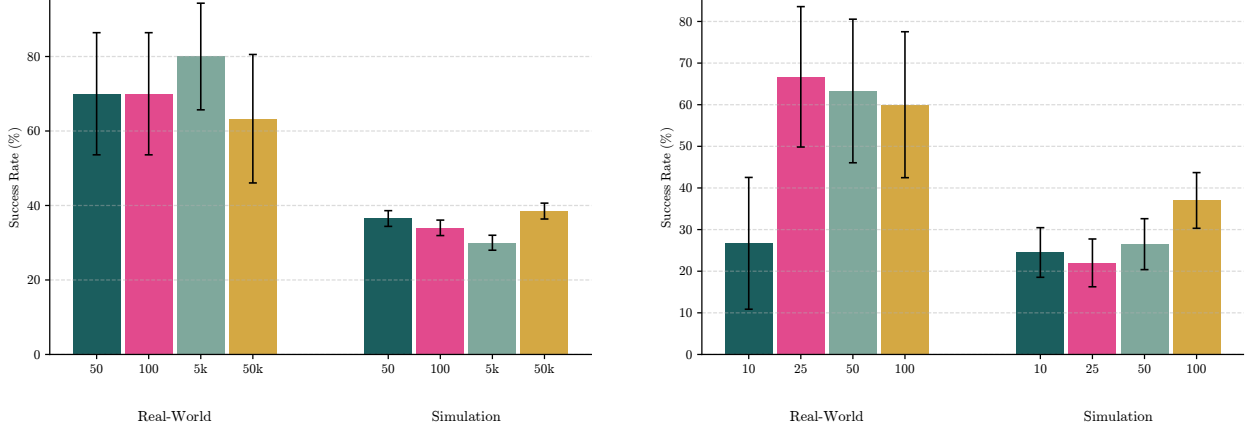
Figure 8 Effect of data scaling on performance. We evaluate MolmoBot-Img trained with differing number of demonstrations sampled from 5,000 houses and report success rate on both DROID (real-world) and the pick classic (simulation). We observe that performance predictably improves with scale, particularly in the real world evaluations. Error bars denote 95% confidence intervals computed from the binomial proportion.

5.2.1 RB-Y1 Results

Table 8 reports zero-shot simulation performance across all RB-Y1 policies. MolmoBot Multitask outperforms MolmoBot-SPOC across all shared tasks, which we attribute to several factors. First, MolmoBot’s frozen VLM backbone provides rich visual representations that generalize well without task-specific finetuning, whereas MolmoBot-SPOC’s smaller transformer architecture has more limited capacity. Second, MolmoBot Multitask was trained jointly across all tasks, which may have enabled positive transfer between related manipulation behaviors. Although MolmoBot-SPOC demonstrates more modest performance in these evaluations, its compact scale enables future on-policy reinforcement learning in simulation, which has been shown to yield substantial performance gains [45].

5.3 Data Ablations

In this section, we study how several properties of the training data affect performance, including data scale, the number of unique objects, the number of unique houses, and image augmentation. We observe some expected trends, such as performance improving monotonically as the amount of training data increases. We also find some surprising results, such as increasing the number of unique house environments having little effect on performance. All data ablations report performance on the pick task for the MolmoBot-Img model trained for 24K steps with batch size 512. The real world evaluations are done with DROID in the workroom



(a) We train MolmoBot-Img on 50,000 pick trajectories drawn from training sets with increasing numbers of simulated house environments. We evaluate the models on DROID and pick classic. We find that unexpectedly increasing the diversity of environments does not lead to improved performance in simulation or real. We hypothesize that due to the local nature of the pick task, diversity in visual background content is unnecessary. Error bars denote 95% confidence intervals computed from the binomial proportion.

(b) We train MolmoBot-Img on 50,000 trajectories drawn from 5,000 houses while controlling the number of unique objects used in the pick demonstrations. We find that although scaling the number of unique objects reliably improves performance in simulation, we do not observe the same trend in the real-world evaluation. We hypothesize that this may be due to the real-world evaluation having significantly fewer and more general objects such as banana and cup.

Figure 9 Ablations on training data diversity. **(a)** Environment diversity scaling. **(b)** Object class diversity scaling.

setup and simulated evaluations are done with Pick Classic as detailed in section 5.2. Further details and analysis can be found below and in the Appendix.

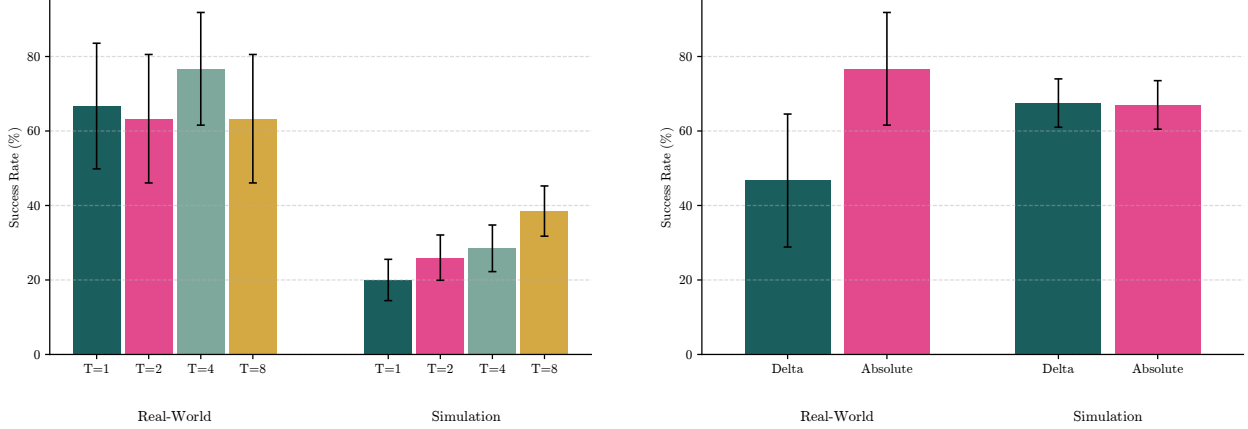
Scaling Number of Demonstrations To study how performance scales with the data scale, we vary the total number of training demonstrations while keeping the number of house environments and object classes fixed. Concretely, we train MolmoBot-Img on datasets containing 10K, 25K, and 50K demonstrations sampled from the same set of 5K environments and 12.4K object categories. We evaluate the model for the pick task in both simulation and real. We observe predictable scaling trends as pick performance for both domains improves with the number of demonstrations (figure 8).

Scaling Environment Diversity For this ablation we vary the number of unique houses in the training set while keeping the total number of demonstrations fixed. Specifically, we contrast many demonstrations from fewer houses with fewer demonstrations spread across more houses. We fix the data scale at 50K trajectories. Unexpectedly, we find that increasing the number of unique training environments has little effect on downstream performance (Figure 9a). This suggests that, for the pick task, performance is driven more by the total amount of interaction data than by scaling environment diversity.

Scaling Object Diversity We train MolmoBot-Img with a fixed number of 50K trajectories while sampling from 5 to 100 objects. We find that performance improves as expected for the simulated evaluation (Figure 9b). However, the performance on DROID does not have a clear trend with respect to object diversity. We hypothesize that the number of objects beyond a small number does not improve performance on DROID because the set of objects in the evaluation is limited and quite general such as apple and cup.

5.4 Model Ablations

Timesteps sampled during training We sampled multiple time steps T per example and denoise in parallel during to improve the convergence and the accuracy of the model. We ablate the choice for $T \in \{1, 2, 4, 8\}$



(a) We train MolmoBot-Img while sampling multiple denoising timesteps T per example in parallel during training to improve convergence and final performance. We ablate $T \in \{1, 2, 4, 8\}$ and evaluate on both DROID and pick classic. We find that simulation performance improves steadily as T increases and peaks at $T = 8$, while real-world performance is less monotonic and peaks at $T = 4$. Error bars denote 95% confidence intervals computed from the binomial proportion.

(b) We train MolmoBot-Img using either absolute or delta action representations for 200K steps on Franka FR3 policies. We find that the absolute action representation substantially improves real-world performance over delta actions when evaluated in our workroom setting, while the two representations perform similarly in simulation. Error bars denote 95% confidence intervals computed from the binomial proportion.

Figure 10 Ablations on training and action parameterization. **(a)** Number of denoising timesteps sampled during training. **(b)** Absolute versus delta action representations.

during training and report the performance of MolmoBot-Img (Figure 10a). Performance on simulation benchmarks improves as T increases and peaks at $T = 8$, suggesting the increase T helps. However, while the result on real subset of 30 examples is not as clear, with the performance peaking at $T = 4$.

Action representation. We compare MolmoBot-Img trained using absolute and delta representations on the complete multi-task data mix for 200K steps each for the Franka FR3 policies (Figure 10b). On the Franka FR3 task, the absolute policy significantly outperforms the delta policy in real setting, while the simulation results are on-par for both policies. The significant gap in real across 3 of our benchmarks strongly suggests that absolute joint policy models transfer better to real world tasks.

6 Conclusion

In this work, we demonstrate that zero-shot transfer to the real world is not only possible, but effective for both static and mobile manipulation. We introduce MolmoBot-Engine, use it to generate MolmoBot-Data and then use that to train three policy classes: MolmoBot, MolmoBot-Pi0 and MolmoBot-SPOC. We evaluate on two robotic platforms: the Franka FR3 for tabletop manipulation tasks and the Rainbow Robotics RB-Y1 mobile manipulator. Without any real-world fine-tuning, our policies achieve zero-shot transfer to unseen objects and environments. On tabletop pick-and-place, MolmoBot achieves a success rate of 79.2% in real world evaluations demonstrating that procedural environment generation combined with diverse articulated assets can produce robust manipulation policies that generalize broadly to the real world. We release MolmoBot-Engine to enable the community to extend this approach to new robots, tasks, and object categories.

While a promising step for scaling up simulation-based pre-training, there are still several avenues of improvement for MolmoBot. MolmoBot-Engine is fundamentally constrained by assets that can currently be accurately simulated. We focus on rigid body and articulated object manipulation—tasks where modern simulators provide sufficient fidelity for transfer. Extending to contact-rich manipulation (e.g., insertion, peg-in-hole), deformable objects (cloth, rope, food), or tasks requiring accurate fluid or granular dynamics remains an open challenge. We believe that coupled with advances in physics-based and generative *world*

model simulators, our recipe of massive-scale procedural generation may extend to these more challenging tasks requiring contact-rich dexterity and deformables. We are excited by the potential frontiers of open research this work enables.

Author Contributions

This project was made possible through the equal contributions of all four co-first authors, who are listed in alphabetical order.

Abhay Deshpande led data generation for Franka FR3, efforts around MolmoBot-Pi0 training, and real evaluation for Franka FR3.

MayaGuru led data generation for RB-Y1 object manipulation + drawer/cabinet articulation, MolmoBot-SPOC architecture + training for mobile manipulation, and real evaluation for RB-Y1.

Rose Hendrix led the project, wrote large parts of the core data engine and scaling infrastructure, and advised on all components.

Snehal Jauhri led data generation for RB-Y1 door opening and MolmoBot training for mobile manipulation.

Ainaz Eftehar led MolmoBot-SPOC architecture and training for static manipulation.

Rohun Tripathi led MolmoBot single and multi-frame architecture and training, focusing on static manipulation.

Jordi Salvador assisted with data scaling, infrastructure, and simulation evaluation.

Max Argus assisted with simulation evaluations and benchmark creation.

Matthew Wallingford led the ablation training and evaluation.

Haoquan Fang contributed the base Molmo2+DiT architecture and the initial flow matching training pipeline.

Wilbert Pumacay assisted with OOD simulation evaluation and additional benchmarks.

Yejin Kim assisted with RB-Y1 real evaluation and data generation.

Quinn Pfeifer, Ying-Chun Lee, Piper Wolters, Omar Rayyan, Mingtong Zhang, and **Jiafei Duan** assisted with simulation and real evaluation.

Karen Farley managed the project.

Winson Han and **Eli Vanderbilt** designed the figures in this report and helped with visualizations.

Dieter Fox, Ali Farhadi and **Georgia Chalvatzaki** advised the project.

Dhruv Shah and **Ranjay Krishna** were co-PIs for the project.

References

- [1] NVIDIA, :, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*, 2025.
- [4] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas,

- Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [5] Kiana Ehsani, Tanmay Gupta, Rose Hendrix, Jordi Salvador, Luca Weihs, Kuo-Hao Zeng, Kunal Pratap Singh, Yejin Kim, Winson Han, Alvaro Herrasti, et al. Spoc: Imitating shortest paths in simulation enables effective navigation and manipulation in the real world. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16238–16250, 2024.
 - [6] Yejin Kim, Wilbert Pumacay, Omar Rayyan, Max Argus, Winson Han, Eli VanderBilt, Jordi Salvador, Abhay Deshpande, Rose Hendrix, Snehal Jauhri, Shuo Liu, Nur Muhammad Mahi Shafiullah, Maya Guru, Arjun Guru, Ainaz Eftekhari, Karen Farley, Donovan Clay, Jiafei Duan, Piper Wolters, Alvaro Herrasti, Ying-Chun Lee, Georgia Chalvatzaki, Yuchen Cui, Ali Farhadi, Dieter Fox, and Ranjay Krishna. Molmospaces: A large-scale open ecosystem for robot navigation and manipulation, 2026.
 - [7] Christopher Clark, Jieyu Zhang, Zixian Ma, Jae Sung Park, Mohammadreza Salehi, Rohun Tripathi, Sangho Lee, Zhongzheng Ren, Chris Dongjoo Kim, Yinyu Yang, Vincent Shao, Yue Yang, Weikai Huang, Ziqi Gao, Taira Anderson, Jianrui Zhang, Jitesh Jain, George Stoica, Winson Han, Ali Farhadi, and Ranjay Krishna. Molmo2: Open weights and data for vision-language models with video understanding and grounding. *arXiv preprint arXiv:2601.10611*, 2026.
 - [8] Dean A. Pomerleau. Alvin, an autonomous land vehicle in a neural network, 2015. URL <https://api.semanticscholar.org/CorpusID:18420840>.
 - [9] Tianhao Zhang, Zoe McCarthy, Owen Jow, Dennis Lee, Ken Goldberg, and P. Abbeel. Deep imitation learning for complex manipulation tasks from virtual reality teleoperation. In *IEEE International Conference on Robotics and Automation*, 2017. URL <https://api.semanticscholar.org/CorpusID:3720790>.
 - [10] Corey Lynch, Mohi Khansari, Ted Xiao, Vikash Kumar, Jonathan Tompson, Sergey Levine, and Pierre Sermanet. Learning latent plans from play. In *Conference on Robot Learning*, 2019. URL <https://api.semanticscholar.org/CorpusID:67877011>.
 - [11] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44:1684 – 1704, 2023. URL <https://api.semanticscholar.org/CorpusID:257378658>.
 - [12] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J. Joshi, Ryan C. Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael S. Ryoo, Grecia Salazar, Pannag R. Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Anand Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Ho Vuong, F. Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale. *ArXiv*, abs/2212.06817, 2022. URL <https://api.semanticscholar.org/CorpusID:254591260>.
 - [13] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Krzysztof Choromanski, Tianli Ding, Danny Driess, Kumar Avinava Dubey, Chelsea Finn, Peter R. Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil J. Joshi, Ryan C. Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Sergey Levine, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael S. Ryoo, Grecia Salazar, Pannag R. Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Ho Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Ted Xiao, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *ArXiv*, abs/2307.15818, 2023. URL <https://api.semanticscholar.org/CorpusID:260293142>.
 - [14] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyu Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *ArXiv*, abs/2510.10274, 2025. URL <https://api.semanticscholar.org/CorpusID:282057092>.
 - [15] Lihan Zha, Asher J. Hancock, Mingtong Zhang, Tenny Yin, Yixuan Huang, Dhruv Shah, Allen Z. Ren, and Anirudha Majumdar. Lap: Language-action pre-training enables zero-shot cross-embodiment transfer, 2026. URL <https://arxiv.org/abs/2602.10556>.

- [16] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew Wang, Anikait Singh, Animesh Garg, Aniruddha Kembhavi, Annie Xie, Anthony Brohan, Antonin Raffin, Archit Sharma, Arefeh Yavary, Arhan Jain, Ashwin Balakrishna, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Blake Wulfe, Brian Ichter, Cewu Lu, Charles Xu, Charlotte Le, Chelsea Finn, Chen Wang, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Christopher Agia, Chuer Pan, Chuyuan Fu, Coline Devin, Danfei Xu, Daniel Morton, Danny Driess, Daphne Chen, Deepak Pathak, Dhruv Shah, Dieter Buehler, Dinesh Jayaraman, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Ethan Foster, Fangchen Liu, Federico Ceola, Fei Xia, Feiyu Zhao, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Gilbert Feng, Giulio Schiavi, Glen Berseth, Gregory Kahn, Guanzhi Wang, Hao Su, Hao-Shu Fang, Haochen Shi, Henghui Bao, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Huy Ha, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jaimyn Drake, Jan Peters, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jeffrey Wu, Jensen Gao, Jiaheng Hu, Jiajun Wu, Jialin Wu, Jiankai Sun, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jimmy Wu, Jingpei Lu, Jingyun Yang, Jitendra Malik, João Silvério, Joey Hejna, Jonathan Booher, Jonathan Tompson, Jonathan Yang, Jordi Salvador, Joseph J. Lim, Junhyek Han, Kaiyuan Wang, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Black, Kevin Lin, Kevin Zhang, Kiana Ehsani, Kiran Lekkala, Kirsty Ellis, Krishan Rana, Krishnan Srinivasan, Kuan Fang, Kunal Pratap Singh, Kuo-Hao Zeng, Kyle Hatch, Kyle Hsu, Laurent Itti, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Linxi Jim Fan, Lionel Ott, Lisa Lee, Luca Weihs, Magnum Chen, Marion Lepert, Marius Memmel, Masayoshi Tomizuka, Masha Itkina, Mateo Guaman Castro, Max Spero, Maximilian Du, Michael Ahn, Michael C. Yip, Mingtong Zhang, Mingyu Ding, Minh Heo, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Ning Liu, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Osbert Bastani, Pannag R Sanketi, Patrick Tree Miller, Patrick Yin, Paul Wohlhart, Peng Xu, Peter David Fagan, Peter Mitrano, Pierre Sermanet, Pieter Abbeel, Priya Sundareshan, Qiuyu Chen, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Rohan Bajjal, Rosario Scalise, Rose Hendrix, Roy Lin, Runjia Qian, Ruohan Zhang, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Shan Lin, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shuran Song, Sichun Xu, Siddhant Halder, Siddharth Karamcheti, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Subramanian Ramamoorthy, Sudeep Dasari, Suneel Belkhale, Sungjae Park, Suraj Nair, Suvir Mirchandani, Takayuki Osa, Tanmay Gupta, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Thomas Kollar, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Trinity Chung, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiaolong Wang, Xinghao Zhu, Xinyang Geng, Xiyuan Liu, Xu Liangwei, Xuanlin Li, Yao Lu, Yecheng Jason Ma, Yejin Kim, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Yilin Wu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yue Cao, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunchu Zhang, Yunfan Jiang, Yunshuang Li, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zehan Ma, Zhuo Xu, Zichen Jeff Cui, Zichen Zhang, and Zipeng Lin. Open x-embodiment: Robotic learning datasets and rt-x models : Open x-embodiment collaboration0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903, 2024. doi: 10.1109/ICRA57147.2024.10611477.
- [17] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Bajjal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, Vitor Guizilini, David Antonio Herrera, Minh Heo, Kyle Hsu, Jiaheng Hu, Muhammad Zubair Irshad, Donovan Jackson, Charlotte Le, Yunshuang Li, Kevin Lin, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O'Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. Droid: A large-scale in-the-wild robot manipulation dataset, 2024.
- [18] Shengliang Deng, Mi Yan, Songlin Wei, Haixin Ma, Yuxin Yang, Jiayi Chen, Zhiqi Zhang, Taoyu Yang, Xuheng

- Zhang, Wenhao Zhang, Heming Cui, Zhizheng Zhang, and He Wang. Graspvla: a grasping foundation model pre-trained on billion-scale synthetic action data, 2025. URL <https://arxiv.org/abs/2505.03233>.
- [19] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, Xu Shi, Ning Gao, Hangxu Liu, Xuekun Jiang, Zherui Qiu, Feng Yuan, et al. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025.
 - [20] Yifan Yin, Zhen Han, Shivam Aarya, Jianxin Wang, Shuhang Xu, Jiawei Peng, Angtian Wang, Alan L. Yuille, and Tianmin Shu. Partinstruct: Part-level instruction following for fine-grained robot manipulation. *ArXiv*, abs/2505.21652, 2025. URL <https://api.semanticscholar.org/CorpusID:278959694>.
 - [21] Abhishek Joshi, Beining Han, Jack Nugent, Max Gonzalez Saez-Diez, Yiming Zuo, Jonathan Liu, Hongyu Wen, Stamatis Alexandropoulos, Karhan Kayan, Anna Calveri, Tao Sun, Gaowen Liu, Yi Shao, Alexander Raistrick, and Jia Deng. Procedural generation of articulated simulation-ready assets, 2025. URL <https://arxiv.org/abs/2505.10755>.
 - [22] Zhenyu Wu, Angyuan Ma, Xiuwei Xu, Hang Yin, Yinan Liang, Ziwei Wang, Jiwen Lu, and Haibin Yan. Moto: A zero-shot plug-in interaction-aware navigation for general mobile manipulation. *ArXiv*, abs/2509.01658, 2025. URL <https://api.semanticscholar.org/CorpusID:281079068>.
 - [23] Jimmy Wu, William Chong, Robert Holmberg, Aaditya Prasad, Yihuai Gao, Oussama Khatib, Shuran Song, Szymon Rusinkiewicz, and Jeannette Bohg. Tidybot++: An open-source holonomic mobile manipulator for robot learning. In *Conference on Robot Learning*, 2024.
 - [24] Haoyu Xiong, Russell Mendonca, Kenneth Shaw, and Deepak Pathak. Adaptive mobile manipulation for articulated objects in the open world. *ArXiv*, abs/2401.14403, 2024. URL <https://api.semanticscholar.org/CorpusID:267212147>.
 - [25] Ben Eisner, Harry Zhang, and David Held. FlowBot3D: Learning 3D articulation flow to manipulate articulated objects. In *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, June 2022. doi: 10.15607/RSS.2022.XVIII.018.
 - [26] Haoru Xue, Tairan He, Zi Wang, Qingwei Ben, Wenli Xiao, Zhengyi Luo, Xingye Da, Fernando Castañeda, Guanya Shi, Shankar Sastry, Linxi Fan, and Yuke Zhu. Opening the sim-to-real door for humanoid pixel-to-action policy transfer, 2025. URL <https://arxiv.org/abs/2512.01061>.
 - [27] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, Aniruddha Kembhavi, Abhinav Kumar Gupta, and Ali Farhadi. Ai2-thor: An interactive 3d environment for visual ai. *ArXiv*, abs/1712.05474, 2017. URL <https://api.semanticscholar.org/CorpusID:28328610>.
 - [28] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13142–13153, 2022. URL <https://api.semanticscholar.org/CorpusID:254685588>.
 - [29] Eric Kolve, Roozbeh Mottaghi, Daniel Gordon, Yuke Zhu, Abhinav Gupta, and Ali Farhadi. AI2-THOR: an interactive 3d environment for visual AI. *CoRR*, abs/1712.05474, 2017. URL <http://arxiv.org/abs/1712.05474>.
 - [30] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, Helen Oleynikova, Ankur Handa, Fabio Ramos, Nathan Ratliff, and Dieter Fox. curobo: Parallelized collision-free minimum-jerk robot motion generation, 2023. URL <https://arxiv.org/abs/2310.17274>.
 - [31] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
 - [32] AgiBot-World-Contributors. Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
 - [33] Ajay Mandekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021.

- [34] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- [35] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. In *International Conference on Learning Representations (ICLR)*, 2026.
- [36] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [37] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. 2025. URL <https://arxiv.org/abs/2502.14786>.
- [38] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- [39] Physical Intelligence. openpi. URL <https://github.com/Physical-Intelligence/openpi>.
- [40] Arhan Jain et al. Polaris: Scalable real-to-sim evaluations for generalist robot policies, 2025. URL <https://arxiv.org/abs/2512.16881>.
- [41] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. 2023. URL <https://arxiv.org/abs/2303.15343>.
- [42] Shengliang Deng, Mi Yan, Yixin Zheng, Jiayi Su, Wenhao Zhang, Xiaoguang Zhao, Heming Cui, Zhizheng Zhang, and He Wang. Stereovla: Enhancing vision-language-action models with stereo vision, 2025. URL <https://arxiv.org/abs/2512.21970>.
- [43] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [44] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies, 2026. URL <https://arxiv.org/abs/2602.15922>.
- [45] Jiaheng Hu, Rose Hendrix, Ali Farhadi, Aniruddha Kembhavi, Roberto Martin-Martin, Peter Stone, Kuo-Hao Zeng, and Kiana Ehsani. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. 2024. URL <https://arxiv.org/abs/2409.16578>.

A Appendix

A.1 Motion Planning

For motion planning for the RB-Y1 robot, we use the Curobo [30] motion generator, specifically, the GPU-accelerated collision-aware trajectory generator. We model the whole-body kinematics as a 23-DOF chain: a 3-DOF holonomic base (two virtual prismatic joints for planar translation and one continuous joint for yaw), a 6-DOF torso, and two 7-DOF arms. Given a target end-effector pose, cuRobo first solves inverse kinematics using 64 seeds, then computes a collision-free trajectory using 4 trajectory optimization seeds with fixed iterations, and finally smooths and interpolates the trajectory to match the simulation control frequency. The robot geometry is approximated by collision spheres, while scene obstacles use mesh-based collision checking with a 0.2m activation distance. Self-collision optimization is enabled, and planning is attempted up to 5 times per query.

A.2 Referral Expressions

In order to sample referral expressions, we first consider task contexts. For example, in a task where the robot is in front of a workbench with some objects on top of it, the context should be the set of objects lying on the workbench surface). In that context, we seek to maximize the contrast between (1) the CLIP similarity between the normalized textual embedding of the referral expression and the normalized visual embedding of the target object, and (2) the CLIP similarity between the referral expression and any of the other objects on the workbench surface. In other words, we do not take into account whether the referral expression would be a better fit for any other object in the scene, as it is the robot’s task to infer the correct context given the task setup and the instruction.

The set of possible referral expressions is composed of LLM-generated short descriptions (between 1 and 5 words), synset lemmas, and normalized object category names. We filter valid expressions that produce a CLIP-similarity contrast ≥ 0.03 while providing a CLIP similarity ≥ 0.1 and sample via a softmax distribution with temperature $2 \cdot 10^{-2}$ over the CLIP similarity contrasts for each filtered expression.

A.2.1 Train-Time Task Prompt Randomization

To further boost the diversity of the language instructions, we procedurally randomize the task prompt during training.

Template Randomization There are many ways to give the same instruction to the policy. Therefore, at train-time, when sampling data, we sample one of several task prompt templates with varying wording and phrasing for use. For brevity, we defer the full list of task prompt templates to the released code.

Referral Expression Randomization MolmoBot-Data saves multiple valid referral expressions for each task-relevant object for each episode. During training, we then sample a referral expression for each object, biasing towards shorter expressions, which we then insert into the sampled prompt template.

A.3 Real-World Evaluation Details

A.3.1 DROID Evaluation Environments

Below we describe each real-world environment used for DROID evaluations in the real world, pictured in Fig. 5. Specific task prompts for each task are detailed in Tab. 9. We also present the full results of every real-world evaluation trial for each policy on each task in each environment in Tab. 10.

Kitchen The kitchen environment consists of 4 objects (mug, computer mouse, apple, banana) and 2 receptacles (brown bowl, black bowl). The receptacles are placed to the left and right of the robot, while the objects are either placed in between the bowls (“easy” placement) or further from the workspace center, closer to the wrong bowl (“hard” placement). For each of these $4 \times 2 = 8$ tasks, the target receptacle is the brown

bowl. For the final 2 tasks, all objects are placed onto the table, and the policy must put the mug into each of the receptacles.

Workroom The workroom environment consists of 5 objects (tape, wooden spoon, timer, copper mug, blue mug) and 2 receptacles (tray, box) on the left of the workspace. Each object must be placed into each receptacle for a total of 10 tasks. When evaluating on the mugs or timer, these objects are placed together on the table in the middle of the workspace. When evaluating on the tape or spoon, they are placed along with an additional spork (a distractor) in the middle of the workspace.

Bedroom The bedroom environment consists of 4 objects (pill bottle, lint roller, banana, tennis ball) and 2 receptacles (towel, basket). Each object is placed into each receptacle for $4 \times 2 = 8$ tasks, and for the final 2 tasks the policy must put the banana into each receptacle, but with a cluttered workspace. This environment notably does not feature a table as a support surface, but instead a bed, which tests robustness environment diversity.

Office The office environment features 8 objects (knife, banana, marker, scissors, carrot, screwdriver, computer mouse, mug) and 7 receptacles (cutting board, plate, mug, green bowl, blue bowl, basket, box), with multiple object configurations with varying amounts of clutter and distractors. Evaluations in this environment were conducted in an entirely different institution and geographical location, illustrating MolmoBot policies’ ability to get up and running in completely new settings.

A.4 Simulation Evaluation Details

We run our experiments with the following hyper-parameters. A policy_dt of 66ms, task_horizon 300 steps / 20 seconds (pick tasks) and 600 steps / 40 seconds (for pick-and-place tasks). When using the filament renderer we set the environment illumination to 12000 candela by default.

StereoVLA was trained based on a front-on view of the robot, in order to run StereoVLA we modify our benchmark by moving cameras into this position, we additionally filter out episodes where the target object is not visible. This yields 92 episodes for the Pick Classic task and 91 episodes for the Pick task. Despite these accommodations for StereoVLA the performance remains low.

Task Name	Object-Dataset	Samples	Renderer	Camera
Pick-MSPProc	Thor	1000	MuJoCo	Droid
Pick-Classic	Objaverse	200	MuJoCo	Droid-Light
Pick	Objaverse	200	Filament	Droid-Light
Pick-Random-Cam	Objaverse	200	Filament	Rnd.-Cam.
PnP-Next-To	Objaverse	200	Filament	Droid-Light
PnP-Color	Objaverse	200	Filament	Droid-Light

Benchmark	Task ID(s)	Task Prompt
Workroom	spoon_tray	“put the wooden spoon on the light blue tray”
	spoon_box	“put the wooden spoon in the wooden box”
	tape_tray	“put the blue tape on the light blue tray”
	tape_box	“put the blue tape in the wooden box”
	blue_mug_tray	“put the blue mug on the light blue tray”
	blue_mug_box	“put the blue mug in the wooden box”
	copper_mug_tray	“put the copper mug on the light blue tray”
	copper_mug_box	“put the copper mug in the wooden box”
	timer_tray	“put the green timer on the light blue tray”
	timer_box	“put the green timer in the wooden box”
Kitchen	apple_easy, apple_hard	“put the apple in the brown bowl”
	clutter_brown, mug_easy, mug_hard	“put the mug in the brown bowl”
	banana_easy, banana_hard	“put the banana in the brown bowl”
	mouse_easy, mouse_hard	“put the computer mouse in the brown bowl”
	clutter_black	“put the mug in the black bowl”
Bedroom	pills_towel	“put the pill bottle on the yellow towel”
	pills_basket	“put the pill bottle in the basket”
	roller_towel	“put the lint roller on the yellow towel”
	roller_basket	“put the lint roller in the basket”
	banana_towel, clutter_towel	“put the banana on the yellow towel”
	banana_basket, clutter_basket	“put the banana in the basket”
	ball_towel	“put the tennis ball on the yellow towel”
	ball_basket	“put the tennis ball in the basket”
Office	knife_board	“put the knife on the cutting board”
	banana_plate	“move the toy banana on the plate”
	marker_mug	“put the marker into the mug”
	scissors_bowl	“pick up the scissor and place it inside the bowl”
	carrot_basket	“put the carrot into the basket”
	knife_green_bowl	“pick up the knife and place it inside the green bowl”
	screwdriver_blue_bowl	“put the screwdriver in the blue bowl”
	mouse_blue_bowl	“place the mouse into the blue bowl”
	mug_bowl	“grasp the mug and put it inside the bowl”
	marker_box	“pick up the red marker and place it into the box”

Table 9 Task prompts for each task in each benchmark for our real-world DROID evaluations.

Benchmark	Policy	<i>Spoon Tray</i>	<i>Spoon Box</i>	<i>Tape Tray</i>	<i>Tape Box</i>	<i>Blue Mug Tray</i>	<i>Blue Mug Box</i>	<i>Copper Mug Tray</i>	<i>Copper Mug Box</i>	<i>Timer Tray</i>	<i>Timer Box</i>	Avg
Workroom	MolmoBot	3/3	2/3	3/3	3/3	3/3	3/3	3/3	1/3	3/3	3/3	90%
	MolmoBot-Img	3/3	3/3	3/3	3/3	3/3	2/3	2/3	0/3	3/3	1/3	77%
	MolmoBot-Pi0	1/3	0/3	3/3	3/3	3/3	3/3	2/3	1/3	0/3	2/3	60%
	$\pi_{0.5}$ -DROID	2/3	2/3	2/3	0/3	1/3	1/3	0/3	0/3	0/3	0/3	27%
	π_0	0/3	0/3	1/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	3%
		<i>Apple Easy</i>	<i>Apple Hard</i>	<i>Mug Easy</i>	<i>Mug Hard</i>	<i>Banana Easy</i>	<i>Banana Hard</i>	<i>Mouse Easy</i>	<i>Mouse Hard</i>	<i>Clutter Brown</i>	<i>Clutter Black</i>	Avg
Kitchen	MolmoBot	1/3	2/3	3/3	3/3	3/3	3/3	3/3	3/3	0/3	0/3	70%
	MolmoBot-Img	3/3	3/3	2/3	3/3	3/3	3/3	3/3	2/3	3/3	1/3	87%
	MolmoBot-Pi0	2/3	3/3	2/3	0/3	3/3	1/3	3/3	2/3	0/3	0/3	53%
	$\pi_{0.5}$ -DROID	3/3	0/3	2/3	1/3	3/3	2/3	3/3	1/3	2/3	2/3	63%
	π_0	0/3	1/3	0/3	0/3	1/3	0/3	3/3	1/3	0/3	0/3	20%
		<i>Pills Towel</i>	<i>Pills Basket</i>	<i>Roller Towel</i>	<i>Roller Basket</i>	<i>Banana Towel</i>	<i>Banana Basket</i>	<i>Ball Towel</i>	<i>Ball Basket</i>	<i>Clutter Towel</i>	<i>Clutter Basket</i>	Avg
Bedroom	MolmoBot	3/3	3/3	3/3	0/3	3/3	2/3	3/3	3/3	3/3	3/3	87%
	MolmoBot-Img	0/3	0/3	1/3	2/3	3/3	3/3	3/3	2/3	3/3	3/3	67%
	MolmoBot-Pi0	2/3	0/3	2/3	0/3	0/3	0/3	1/3	0/3	0/3	2/3	23%
	$\pi_{0.5}$ -DROID	0/3	0/3	0/3	0/3	1/3	0/3	0/3	0/3	2/3	0/3	10%
	π_0	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0/3	0%
		<i>Knife Board</i>	<i>Banana Plate</i>	<i>Marker Mug</i>	<i>Scissors Bowl</i>	<i>Carrot Basket</i>	<i>Knife Green Bowl</i>	<i>Screwdriver Blue Bowl</i>	<i>Mouse Blue Bowl</i>	<i>Mug Bowl</i>	<i>Marker Box</i>	Avg
Office	MolmoBot	2/3	3/3	1/3	2/3	2/3	2/3	3/3	2/3	3/3	1/3	70%
	MolmoBot-Img	2/3	1/3	0/3	1/3	3/3	1/3	3/3	2/3	3/3	2/3	60%
	MolmoBot-Pi0	1/3	3/3	0/3	0/3	0/3	1/3	3/3	2/3	3/3	2/3	50%
	$\pi_{0.5}$ -DROID	2/3	3/3	1/3	1/3	1/3	1/3	2/3	1/3	3/3	2/3	57%
	π_0	0/3	0/3	0/3	0/3	1/3	1/3	1/3	1/3	0/3	0/3	13%

Table 10 DROID real-world evaluation results on each task in each environment. Each cell shows successes out of 3 trials.